

SaG-Prescribe: Unifying Diagnostic Pathways with Counterfactual Refactoring for Automated Code Review and Software Quality Evaluation in Distributed Publish–Subscribe Systems

Author Information Redacted for Double-Blind Review

Special Issue: Intelligent Techniques for Automated Code Review and Software Quality Evaluation
Automated Software Engineering (Springer)

August 30, 2026

Abstract

Automated code review and software quality evaluation are vital for modern software engineering, where systems evolve rapidly and continuous integration demands proactive quality assurance. However, conventional quality assurance practices rely primarily on modular static code analysis (SCA) operating at the file or class level. In distributed publish–subscribe architectures (ROS 2, DDS, MQTT), loose coupling among producers and consumers obscures indirect dependencies, creating an **Architecture-Code Gap**: a codebase may exhibit clean unit-level metrics while harboring severe architectural technical debt, such as single points of failure, co-located deployment bottlenecks, and transport contract mismatches. Furthermore, existing structural diagnostic frameworks operate open-loop—ranking components by scalar criticality without identifying the underlying architectural pathology, attributing it to an engineering role, or verifying remediation actions.

To address these challenges, we introduce **SaG-Prescribe**, an intelligent static system analysis framework that brings a deterministic **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation. The framework unifies two complementary mechanisms: (1) **Explainable Diagnostic Pathway & Technical Debt Analysis**: It extracts a multi-layer heterogeneous dependency graph from Architecture-as-Code descriptors, synthesizes code-level SCA metrics into a **Code Quality Penalty (CQP)**, and computes a hierarchical **Reliability–Maintainability (RM)** quality attribution model grounded in ISO/IEC 25010 and ISO/IEC 25019 Quality-in-Use standards. It drives a catalog of nineteen formal publish–subscribe anti-patterns and bad smells with adaptive box-plot thresholds, providing explainable review decisions mapped to specific engineering roles (Reliability Engineers, SREs, Software Architects). (2) **Prescriptive Refactoring with Counterfactual Verification**: It compiles diagnostic findings into three graph-mutation refactoring operators (logical topic splitting, physical anti-affinity reallocation, and transport QoS contract hardening). Crucially, every candidate refactoring is independently verified on a sandboxed counterfactual graph against a discrete-event cascade simulator, admitting only edits whose measured risk reduction exceeds simulator seed noise across a sweep of propagation thresholds ($\overline{\Delta I} > \kappa \sigma_{\text{seed}}$).

We evaluate SaG-Prescribe across eight benchmark scenarios spanning six architectural domains (autonomous vehicles, IoT smart cities, financial trading, healthcare, hub-and-spoke microservices, and hyper-scale enterprise systems, 29–3325 vertices), reporting empirical findings with full methodological transparency, including two calibration results a purely accuracy-framed evaluation would omit. On the diagnostic side, composite criticality, betweenness, and degree centrality all saturate around NDCG@10 $\approx$ 0.82–0.86 and Cohen’s $\kappa \approx$ 0.41–0.48 against cascade-impact ground truth (mean Spearman $\rho =$ 0.485 for the composite, app layer, versus 0.519 for degree alone)—no single scalar ranking dominates, which is precisely why deterministic, role-attributed diagnosis (Reliability Engineer vs. SRE

vs. Architect) is the pathway’s value, not superior ranking accuracy. This re-measures, under an independent discrete-event cascade oracle, a criticality-ranking claim from our own prior work ($\rho = 0.94$ on a ten-application system against reachability loss); we report the full methodological delta (§1.5, §9.3). We also find that the anti-pattern catalog, while a highly sensitive component-level screen (mean recall 0.942, precision 0.253, implicating 94.8% of components), agrees with ground-truth criticality at chance level (Cohen’s $\kappa = -0.002$)—a consequence of 98.4% of its 6329 findings (dominated by two edge-scoped patterns) not being attributable to any single component—and that the Availability dimension is measured on a degenerate substrate at this layer (zero predicted articulation points in 6 of 8 scenarios, SPOF detector $F_1 = 0.0$ throughout), motivating threshold recalibration, delta-aware gating, and layer-aware scoring. We also document a Simpson’s paradox pooling effect across heterogeneous node types ($\rho = 0.028$ pooled vs. $\rho = 0.503$ on applications). On the prescriptive side, counterfactual verification admits 1128 of 1589 candidate refactorings (71.0%), with anti-affinity reallocation demonstrating the highest survival rate (77.7%), followed by transport QoS hardening (58.0%) and topic splitting (49.7%). Every scenario achieves a statistically significant net reduction in System Risk Index (Wilcoxon $W = 0, p = 0.0156, n = 7$). The diagnostic review gate executes in 0.01–20.98 s, confirming its feasibility as a real-time, pre-commit automated code review check in CI/CD pipelines.

Keywords: Automated code review, software quality evaluation, diagnostic pathway, architectural anti-patterns, bad smells, technical debt analysis, refactoring recommendation, publish–subscribe middleware, CI/CD quality gates, counterfactual verification.

1 Introduction

1.1 Context and Motivation: The Need for Architectural Code Review

Automated code review and software quality evaluation have become indispensable pillars of modern software engineering [1]. As distributed software systems scale in complexity and release cycles compress under continuous integration and continuous delivery (CI/CD) regimes, automated tools must detect regressions, evaluate non-functional requirements, and recommend actionable remediations before defective changes reach production fabrics.

Distributed publish–subscribe middleware frameworks—including the Robot Operating System (ROS 2), the Object Management Group’s Data Distribution Service (DDS), and MQTT—form the communication infrastructure of contemporary microservices, IoT smart cities, and safety-critical cyber-physical systems [2, 3]. These frameworks decouple message producers and consumers across space, time, and synchronization via topic-based publish–subscribe channels and message brokers. However, this architectural decoupling introduces an insidious challenge for automated quality assurance: it obscures the end-to-end dependency structure of the system. Failures, buffer overflows, and latency spikes propagate along indirect topological pathways that are completely invisible when analyzing source code files in isolation.

1.2 Two Open Gaps: The Architecture-Code Gap and Open-Loop Review

Quality evaluation in continuous integration has historically relied on Static Code Analysis (SCA) platforms (e.g., SonarQube, SpotBugs, ESLint). This creates a fundamental **Architecture-Code Gap**:

1. **The Architecture-Code Gap:** A distributed system can achieve pristine code quality within every individual component—clean class hierarchies, zero file-level code smells, and low cyclomatic complexity—yet remain brittle at the architectural topology level. Vulnerabilities such as single points of failure (SPOFs), severe topic fan-out bottlenecks, co-located deployment risks, and incompatible Quality of Service (QoS) transport contracts cannot be detected by examining source files in isolation. Shifting structural dependability

checks left into the development lifecycle demands a transition from traditional Static Code Analysis to *Static System Analysis (SSA)*.

2. **The Unnamed-Pathology and Open-Loop Review Gap:** In object-oriented programming, mature catalogs of bad smells and anti-patterns (e.g., God Class, Feature Envy, Shotgun Surgery) provide developers with a shared vocabulary, testable detection rules, and proven refactoring patterns [4–6]. Microservices research has similarly begun cataloging smells (e.g., distributed monoliths, cyclic dependencies, shared databases) [7]. In contrast, *publish–subscribe architectures lack an equivalent formal anti-pattern catalog*. Consequently, architectural flaws are discovered reactively through production cascade postmortems rather than proactively during code review. Moreover, existing topology-aware diagnostic frameworks typically operate *open-loop*: they compute abstract numerical centrality scores or black-box failure-impact predictions without identifying the specific architectural pathology at fault or synthesizing verified, actionable refactoring guidance [8, 9].

1.3 The Diagnostic Pathway: Transparent, Explainable Software Quality Evaluation

To address these gaps, this paper brings the **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation in distributed systems. We present **SaG-Prescribe**, a comprehensive framework that integrates deterministic diagnostic quality attribution, anti-pattern detection, and closed-loop prescriptive refactoring within continuous integration workflows.

SaG-Prescribe decouples quality assurance into two principled, cooperative pathways (Figure 1):

- **The Diagnostic Pathway (Deterministic Software Quality Evaluation & Attribution):** The Diagnostic Pathway operates statically on Architecture-as-Code descriptors. It constructs a multi-layer heterogeneous dependency graph (G_{analysis}) encompassing applications, libraries, topics, brokers, and physical hosting nodes. It bridges code-level and architecture-level quality by synthesizing modular SCA metrics into a **Code Quality Penalty (CQP)**. It then applies a hierarchical **Reliability–Maintainability (RM)** attribution model grounded in the **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) standards [10, 11]. Reliability decomposes into **Fault Tolerance (FT)**, error propagation reach) and **Availability (A)**, structural single-point-of-failure exposure), while **Maintainability (M)** quantifies coupling complexity and change ripple risk. The diagnostic pathway maps structural technical debt directly to stakeholder harm categories (**h_{QIU}**) and routes explainable findings to specific engineering roles (Reliability Engineers, DevOps/SREs, and Software Architects). Over these metric vectors, it executes a catalog of nineteen formal publish–subscribe anti-patterns using adaptive box-plot thresholds. Crucially, its contribution is *attribution*, not superior ranking accuracy: Section 9.1 shows composite criticality, betweenness, and degree centrality reach comparable ranking quality, so what the pathway adds is a deterministic, auditable decomposition a scalar centrality score cannot provide.
- **The Counterfactual Prescriptive Pathway (Closed-Loop Refactoring Verification):** Rather than offering open-loop suggestions, the prescriptive engine compiles diagnostic findings into three typed graph-mutation operators: logical topic splitting, physical anti-affinity reallocation, and transport QoS contract hardening. It subjects every candidate refactoring to **counterfactual verification** on an in-memory sandbox graph (**MemoryRepository**), simulating failure cascades across multiple seeds and propagation thresholds. An edit is admitted if and only if its measured impact reduction exceeds the

simulator’s seed noise at every threshold ($\overline{\Delta I} > \kappa \sigma_{\text{seed}}$), and the composite policy is gated on a net positive System Risk Index delta ($\Delta \text{SRI} > 0$).

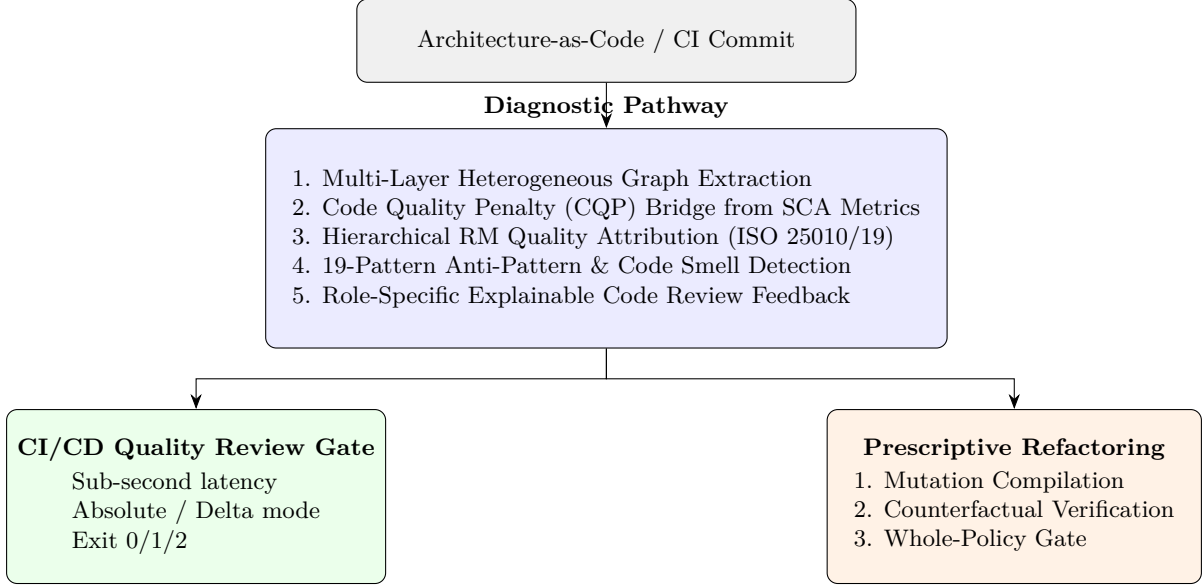


Figure 1: Architectural overview of SaG-Prescribe: the Diagnostic Pathway (Section 3–4) is the shared substrate feeding both the CI/CD quality gate (Section 7) and the counterfactual prescriptive engine (Sections 5–6).

1.4 Contributions

This paper makes the following contributions to automated software quality assurance and code review:

- 1. A Formal Diagnostic Pathway for Pub-Sub Software Quality Evaluation (Section 3):** We formalize a multi-level diagnostic model that ingests file-level SCA metrics via the Code Quality Penalty (CQP), derives multi-layer dependency projections (G_{analysis}), and computes a hierarchical Reliability–Maintainability (RM) attribution model grounded in ISO/IEC 25010 and ISO/IEC 25019 Quality-in-Use standards, including an explicit statement of the layer on which each dimension is well-posed.
- 2. A Comprehensive Catalog of 19 Publish–Subscribe Anti-Patterns and Bad Smells (Section 4):** We formalize nineteen architectural anti-patterns and smells as topological signatures over a structural metric vector with adaptive box-plot thresholds, classify each pattern’s finding scope (component vs. edge vs. system-level), and report which patterns actually trigger on the benchmark corpus.
- 3. A Calibration and Validity Analysis of Deterministic Architectural Diagnosis (Section 9.1–9.3):** Rather than reporting ranking correlation alone, we measure agreement (Cohen’s κ), ranking-quality saturation across three predictors (NDCG@10, Top- k), the mechanism behind the catalog’s over-flagging (edge- vs. component-scoped findings), and the structural degeneracy of the Availability dimension on the application-layer projection—evidence a purely accuracy-framed evaluation would omit.
- 4. A Prescriptive Refactoring Pipeline with Counterfactual Verification (Sections 5–6):** We introduce a rule-based refactoring compiler translating diagnostic findings

into three graph-mutation operators (topic splitting, anti-affinity reallocation, QoS hardening), verified against a discrete-event cascade simulator using a two-level margin-gating criterion. We disclose the exact automation footprint (5 of 19 patterns automated, 14 advisory).

5. **An Automated Code Review Bot and CI/CD Quality Gate (Section 7):** We operationalize the Diagnostic Pathway as a sub-second pre-merge quality gate featuring standardized exit codes (0/1/2) and formalize delta-aware merge-base regression semantics.
6. **Empirical Evaluation and Methodological Findings (Sections 8–9):** Evaluated across eight benchmark scenarios (29 to 3325 vertices; 12 to 300 applications), we report: (a) diagnostic ranking saturates across predictors (mean Spearman $\rho = 0.485$ for the composite vs. 0.519 for degree centrality, both around $\text{NDCG@10} \approx 0.82\text{--}0.86$) and exhibits a Simpson’s paradox pooling effect ($\rho = 0.028$ pooled vs. 0.503 on applications); (b) the catalog is a sensitive but chance-level-agreeing screen (recall = 0.942, precision = 0.253, Cohen’s $\kappa = -0.002$, implicating 94.8% of components, 98.4% of findings not component-scoped); (c) the Availability dimension is degenerate at the application layer (SPOF $F_1 = 0.0$ in all 8 scenarios); (d) counterfactual verification admits 1128 of 1589 candidates (71.0%), with anti-affinity reallocation demonstrating 77.7% survival, and every scenario achieving statistically significant risk reduction (Wilcoxon $W = 0, p = 0.0156, n = 7$); (e) diagnostic review gating executes in 0.01–20.98 s, confirming real-time CI/CD viability.

1.5 Relationship to Prior Work

This submission relates to two prior bodies of work along different axes.

First, our earlier conference paper [12] introduced the multi-layer pub-sub graph model this paper builds on and a two-term Criticality Score, $CS(v) = 0.7 \cdot C_B(v) + 0.3 \cdot AP(v)$ (betweenness and articulation-point membership), validated on a ten-application simulated system against Reachability Loss—the fraction of directed application-pair paths broken by removing v —reporting $\rho = 0.94$. That validation shares its substrate with its predictor: Reachability Loss is itself a connectivity statistic on the same undirected application graph that C_B and AP are computed on, and the benchmark is a single, small system. This paper re-measures the analogous claim under an independent discrete-event cascade oracle (I_{comp} , Section 4.5) across eight scenarios up to 300 applications, on the directed `DEPENDS_ON` projection rather than the undirected pub-sub graph, and finds a substantially lower correlation ($\rho = 0.485$) that is not surpassed by any single structural predictor (Section 9.1). We report the full delta with complete candor, including the mechanism (Section 9.3): on the projection this paper measures, the application layer has essentially no articulation points, so $AP(v)$ —and hence 30% of the prior work’s own composite—carries almost no signal at this layer, an instance of the more general substrate-adequacy issue Section 3.5 states as a modeling constraint rather than an ex post excuse.

Second, our companion manuscript [13] introduces the heterogeneous graph schema, simulation kernels, and a learned Graph Neural Network (GNN) for failure-impact prediction; this paper contributes the **Diagnostic Pathway** for automated code review, the Code Quality Penalty (CQP), the 19-pattern pub-sub anti-pattern catalog with its scope and coverage analysis, the prescriptive refactoring compiler, the counterfactual verification engine, the CI/CD quality gate, and the comprehensive empirical evaluation of detection, calibration, and prescription. Detection figures previously reported from preliminary conference workshops are re-measured here with strict artifact reproducibility.

1.6 Organization

Section 2 surveys related work. Section 3 formalizes the Diagnostic Pathway and software quality model. Section 4 presents the anti-pattern catalog. Section 5 defines the closed-loop optimization objective. Section 6 details the prescriptive pipeline. Section 7 describes CI/CD integration. Sections 8 and 9 present the experimental design and results. Section 10 discusses implications and threats to validity, and Section 11 concludes.

2 Background and Related Work

2.1 Publish–Subscribe Middleware Dependability

Message-oriented middleware research has historically focused on protocol correctness, broker replication, network load balancing, and runtime QoS verification. Classical broker fault-tolerance techniques replicate or partition broker processes to withstand crash faults [14, 15], while log-centric architectures such as Apache Kafka provide distributed commit-log durability [16]. In DDS and ROS 2 ecosystems, recent studies investigate the probabilistic latency characteristics of QoS-driven retransmission protocols [2] and static verification of interdependent QoS policies [3]. While Chaos Engineering injects runtime faults into active deployments, it introduces operational hazards and occurs late in the software lifecycle. SaG-Prescribe shifts resilience analysis left into CI/CD, evaluating Architecture-as-Code descriptors statically before deployment.

2.2 Automated Code Review and Static System Analysis

Automated code review tools traditionally focus on lexical syntax, coding conventions, and localized security vulnerabilities via AST parsing and data-flow analysis (e.g., SonarQube, SpotBugs). Recent research has advanced automated code review using deep learning and large language models for review comment generation and code change assessment [1, 17]. However, these approaches remain confined to source-file boundaries. They cannot infer system-wide topological cascades resulting from decentralized pub-sub message routing. SaG-Prescribe introduces *Static System Analysis (SSA)*, augmenting file-level code quality metrics with global architectural graph models to review structural and middleware-level design flaws.

2.3 Software Quality Models and Explainable Diagnostic Pathways

Software quality models, standardized in ISO/IEC 25010 (Product Quality) and ISO/IEC 25019 (Quality-in-Use), provide structured taxonomies decomposing quality into maintainability, reliability, and usability [10, 11]. While these standards establish top-down quality characteristics, mapping them to concrete, automated measurements in distributed architectures remains challenging. Classical software analytics often collapse multiple structural metrics into a single opaque score or rely on black-box machine learning classifiers that lack explanatory transparency [18]. In automated code review, developers reject opaque recommendations without clear causal rationale.

Two distinct notions of “explainability” appear in this literature, and they are not interchangeable. *Post-hoc attribution* methods (e.g., SHAP, LIME, and attention-weight inspection for GNN-based scorers) approximate why an already-trained black-box model produced a given score, but the explanation is itself an approximation, can be unstable across re-fits, and does not constrain the model to respect any declared quality taxonomy. *Score decomposition*, by contrast, is intrinsic: the score is defined as a declared weighted sum over named sub-characteristics, so the same computation that produces $Q(v)$ also produces its provenance. The Diagnostic Pathway in SaG-Prescribe adopts the latter: it decomposes structural and code metrics into explicit Reliability (Fault Tolerance, Availability) and Maintainability dimensions, mapping

technical debt directly to stakeholder harm vectors and specific engineering roles, with no separate explanation model to validate or drift out of sync with the scorer it explains.

2.4 Anti-Pattern and Code Smell Catalogs

In object-oriented software engineering, Fowler’s refactoring catalog [4] and Brown et al.’s AntiPatterns taxonomy [5] formalized bad smells (e.g., God Class, Feature Envy) alongside remediation strategies. Suryanarayana et al. systematized design smells using formal structural metrics [6]. In distributed systems, Richardson cataloged microservices design patterns [19], while Taibi et al. established empirical taxonomies of microservices anti-patterns (e.g., Cyclic Dependency, Shared Database, Distributed Monolith) [7]. Our catalog (Section 4) adopts this proven specification structure—formal topological rule, affected quality dimension, and concrete remediation—while addressing the unique structural anomalies of publish–subscribe middleware (e.g., broker saturation, topic fan-out explosion, QoS policy mismatches).

2.5 Refactoring Recommendation and Technical Debt Management

Automated refactoring recommendation has been investigated across multiple paradigms:

1. *Metric- and Rule-Based Recommenders*: Rule-driven engines identify code smells and suggest deterministic refactorings [4, 6].
2. *Search-Based Software Engineering (SBSE)*: Multi-objective optimization algorithms (e.g., NSGA-II) search architectural trade-off spaces to optimize coupling and cohesion [8, 9, 20]. However, traditional SBSE operates open-loop, generating candidate architectures without empirically verifying resilience gains against simulated failure cascades.
3. *Machine Learning-Based Refactoring*: Supervised and self-affirming models predict refactoring opportunities from historical commit data and code quality trajectories [21–23].
4. *LLM-Assisted Refactoring*: Recent foundation models assist in automated code transformation and refactoring explanation [24–26].

SaG-Prescribe differs fundamentally in scope and verification: its scope is the global publish–subscribe topology, and its verification model is strictly closed-loop—every candidate refactoring is measured on a counterfactual sandbox graph against a cascade failure simulator before being presented to the architect.

2.6 Structural Criticality and Graph Centrality

Graph-theoretic analysis has long utilized centrality indices—betweenness [27], PageRank, closeness, and articulation points—to identify critical vertices. Bakhtin et al. applied network centralities to microservice call graphs [28], while complex network studies demonstrate scale-free, hub-dominated characteristics in software dependency graphs [29]. Closest to this paper, our own earlier conference work [12] combined betweenness and articulation-point membership into a two-term Criticality Score and validated it against reachability loss on a ten-application pub-sub system, reporting $\rho = 0.94$. Baldwin and Clark’s design structure matrices [30] and combinatorial network reliability theory [31] provide formal foundations for topological fault tolerance. While theoretical literature posits that multi-dimensional metrics should outperform simple centrality on heterogeneous graphs, our empirical findings (Section 9.1)—including a re-measurement of [12]’s own claim under an independent cascade oracle and a larger, more heterogeneous benchmark corpus—reveal that scalar degree centrality remains a formidable baseline and that ranking accuracy alone is not where a diagnostic pathway’s value lies, reinforcing the necessity of empirical verification over theoretical assertion.

3 System Model and The Multi-Level Diagnostic Pathway

3.1 Heterogeneous Graph Formulation

A distributed publish-subscribe system is modeled as a typed, weighted, directed multigraph:

$$G = (V, E, \tau_V, \tau_E, w_E, w_V) \quad (1)$$

where the vertex type mapping $\tau_V : V \rightarrow T_V$ partitions components into five distinct semantic classes:

$$T_V = \{\text{Application, Library, Topic, Broker, Node}\} \quad (2)$$

- **Application (V_{app}):** Active execution processes (e.g., ROS 2 nodes, microservices) that publish or subscribe to data channels.
- **Library (V_{lib}):** Shared software libraries linked by applications.
- **Topic (V_{topic}):** Named communication channels routing message streams.
- **Broker (V_{broker}):** Message routing intermediaries (e.g., MQTT brokers, DDS participants).
- **Node (V_{node}):** Physical or virtual compute hosts providing computational substrate.

The edge type mapping $\tau_E : E \rightarrow T_E$ assigns each link to an architectural relationship:

$$T_E = \{\text{PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES}\} \quad (3)$$

3.2 Derived Dependencies: The DEPENDS_ON Projection

To expose indirect failure propagation channels hidden behind asynchronous pub-sub decoupling, the Diagnostic Pathway derives explicit DEPENDS_ON relations (G_{analysis}) via formal projection rules (directed as dependent $\rightarrow$ dependency):

- **Application-to-Application:** Formed when application A_1 subscribes to a topic T published by application A_2 : $(A_1 \rightarrow A_2) \in E_{\text{depends}}$.
- **Application-to-Broker:** Formed when application A relies on broker B routing topic T : $(A \rightarrow B) \in E_{\text{depends}}$.
- **Application-to-Library:** Formed when application A uses shared library L : $(A \rightarrow L) \in E_{\text{depends}}$, capturing simultaneous blast radius if L experiences a defect.
- **Broker-to-Broker / Node Colocation:** Captures shared-fate host vulnerabilities when multiple brokers or critical services share physical node N .

The projection is organized across four architectural layers:

1. **Application Layer (app):** $V_{\text{app}} \cup V_{\text{lib}}$, focusing on business logic dependencies.
2. **Infrastructure Layer (infra):** V_{node} , capturing compute host topology.
3. **Middleware Layer (mw):** $V_{\text{app}} \cup V_{\text{broker}}$, capturing message mediation.
4. **System Layer (system):** All five vertex types, capturing global cross-layer coupling.

3.3 The Code Quality Penalty (CQP): Bridging SCA and Architecture

To bridge source-code health with architectural risk during automated review, the Diagnostic Pathway ingests four file-level static code analysis (SCA) signals for Application and Library vertices: Lines of Code (`loc`, from `code_metrics.size.total_loc`), Weighted Methods per Class as a complexity proxy (`cyclomatic_complexity`, from `code_metrics.complexity.avg_wmc`), Lack of Cohesion in Methods (`lcom`, from `code_metrics.cohesion.avg_lcom`), and code-level instability, $\text{instability_code}(v) = C_e(v)/(C_a(v) + C_e(v))$, computed from afferent/efferent coupling (`avg_fanin/avg_fanout`). The SonarQube Technical Debt Ratio (`sqale_debt_ratio`) is ingested and persisted alongside these fields but does not enter the CQP formula below or any other scoring path (Table 1); it is exported for dashboard display only.

LOC, complexity, and LCOM are **min-max normalized independently within each node type**—Application and Library vertices form two separate populations, since their typical scales differ—while instability is already bounded in $[0, 1]$ and is not renormalized. A population with zero variance (e.g., every node of a type carries no `code_metrics`) collapses to a normalized value of 0 for all its members rather than 1, so that "uniformly absent data" is not scored as "uniformly worst"; a population of exactly one measured node keeps its full normalized value of 1. These normalized signals synthesize into a per-component **Code Quality Penalty (CQP)**:

$$\begin{aligned} \text{CQP}(v) = & 0.10 \text{loc_norm}(v) + 0.35 \text{complexity_norm}(v) \\ & + 0.30 \text{instability_code}(v) + 0.25 \text{lcom_norm}(v) \end{aligned} \quad (4)$$

$\text{CQP}(v) \in [0, 1]$ provides an explicit, quantitative bridge from source-code technical debt to architectural criticality, entering directly into the Maintainability dimension of the RM model.

Table 1: CQP input fields: what feeds the score vs. what is ingested but does not.

Field	CQP term	Status
<code>loc</code>	$0.10 \times \text{loc_norm}$	Scores
<code>cyclomatic_complexity</code> (avg WMC)	$0.35 \times \text{complexity_norm}$	Scores
<code>lcom</code>	$0.25 \times \text{lcom_norm}$	Scores
<code>avg_fanin/avg_fanout</code>	$0.30 \times \text{instability_code}$	Scores
<code>sqale_debt_ratio</code>	—	Ingested, exported, never scored

3.4 Hierarchical Reliability–Maintainability (RM) Quality Attribution

Grounded in **ISO/IEC 25010** (Product Quality) and **ISO/IEC 25019** (Quality-in-Use) [10,11], the Diagnostic Pathway decomposes structural quality into two primary dimensions—**Reliability** and **Maintainability**—providing explainable, role-specific diagnostics (Figure 2, Table 2).

Table 2: RM Quality Dimensions, Stakeholder Roles, and ISO/IEC 25019 Quality-in-Use Mapping.

Dim.	Sub-Characteristic	Engineering Role	High Score Meaning	ISO 25019 Harm
R	Fault Tolerance (FT)	Reliability Engineer	Cascades deeply; large blast radius	Indirect Harm
	Availability (A)	DevOps / SRE	Single point of failure; partition	Primary Task Loss
M	Structural Coupling	Software Architect	High change ripple; code-level debt	Maintainer Effort

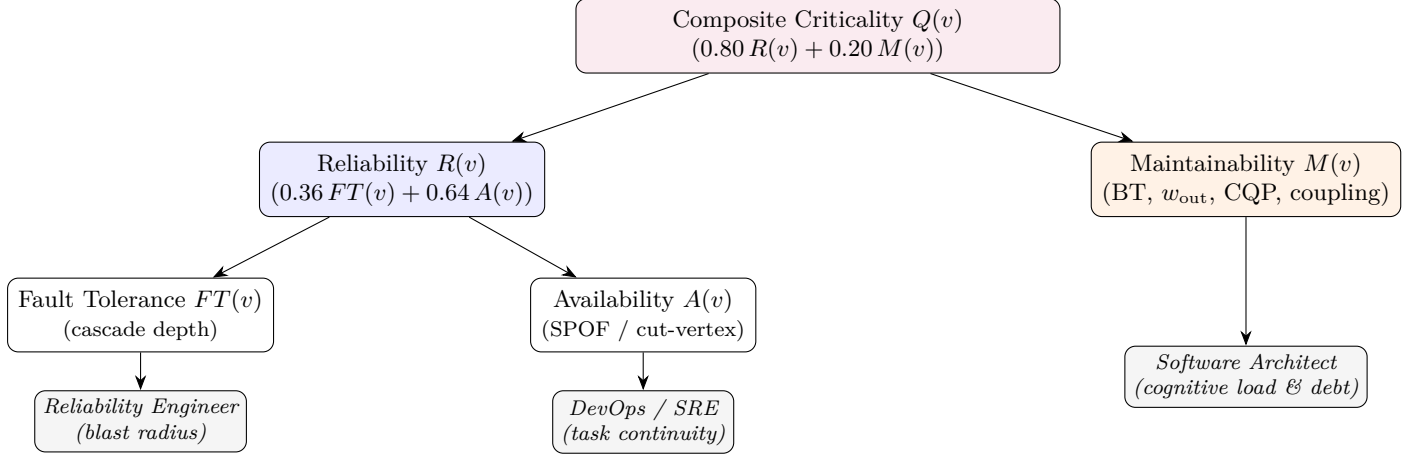


Figure 2: The hierarchical RM Diagnostic Attribution Model and stakeholder role mapping. Each arrow is a scoring dependency computed by the same deterministic pass, not a post-hoc explanation of a separately trained model.

Formal Quality Definitions.

- **Definition D1 (Component Criticality):** Let $G_l = (V_l, E_l)$ be a layer-projected graph. Component criticality $\text{crit}_l : V_l \rightarrow [0, 1]^2 \times [0, 1]$ maps vertex v to metric vector $\mathbf{s}(v) = [R(v), M(v)]^T$ and composite score $Q(v)$, estimating Quality-in-Use degradation.
- **Definition D2 (Relationship Criticality):** Let $e = (u, v) \in E_l$ be a dependency edge. Relationship criticality $\text{crit}_l(e) \in [0, 1]$ estimates Quality-in-Use loss resulting from link disruption.
- **Definition D3 (Consequence Metric):** Criticality measures structural consequence given failure. Failure likelihood must be supplied externally from operational MTTF telemetry.
- **Definition D4 (Relative Distribution):** Criticality classifications are relative to the score distribution of system S at layer l .

Mathematical Formulation of Quality Dimensions.

1. **Fault Tolerance (FT):** Evaluates cascade reach using Reverse PageRank (RPR) and fan-out concentration:

$$FT(v) = f(\text{RPR}(v), \text{DG}_{\text{in}}(v), \text{MPCI}(v), \text{FOC}(v)) \quad (5)$$

2. **Availability (A):** Evaluates single-point-of-failure exposure via directed cut-vertex articulation scoring ($\text{AP}_{c,\text{directed}}$) and bridge ratios:

$$A(v) = g(\text{AP}_{c,\text{directed}}(v), \text{BR}(v), \text{CDI}(v), w(v)) \quad (6)$$

3. **Reliability Composite (R):** Blends Fault Tolerance and Availability with $\alpha = 0.36$:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v) \quad (7)$$

4. **Maintainability (M):** Blends betweenness centrality (BT), efferent QoS out-degree (w_{out}), enhanced coupling risk, and the Code Quality Penalty (CQP):

$$M(v) = 0.35 \text{BT}(v) + 0.30 w_{\text{out}}(v) + 0.15 \text{CQP}(v) + 0.12 \text{CouplingRisk_enh}(v) + 0.08 (1 - \text{CC}(v)) \quad (8)$$

Stakeholder Harm Projection Matrix. The diagnostic vector $\mathbf{s}_{\text{RM}}(v) = [R(v), M(v)]^T$ projects into ISO/IEC 25019 Quality-in-Use stakeholder harm scores $[H_{\text{Ben}}, H_{\text{Risk}}, H_{\text{Acc}}]^T$ via transformation matrix $\mathbf{M}_{\text{RM} \rightarrow \text{QiU}}$:

$$\mathbf{h}_{\text{QiU}}(v) = \mathbf{M}_{\text{RM} \rightarrow \text{QiU}} \cdot \mathbf{s}_{\text{RM}}(v) = \begin{bmatrix} 0.75 & 0.25 \\ 0.80 & 0.20 \\ 0.60 & 0.40 \end{bmatrix} \begin{bmatrix} R(v) \\ M(v) \end{bmatrix} \quad (9)$$

Composite criticality blends the dimensions under declared weights $(w_R, w_M) = (0.80, 0.20)$ [32]:

$$Q(v) = 0.80 R(v) + 0.20 M(v) \quad (10)$$

Components are categorized into five severity tiers (**CRITICAL**, **HIGH**, **MEDIUM**, **LOW**, **MINIMAL**) using adaptive box-plot thresholding ($Q > Q_3 + 1.5 \text{ IQR}$ for **CRITICAL**; $Q_3 < Q \leq \text{upper fence}$ for **HIGH**).

3.5 Substrate Adequacy: Which Layer Each Dimension Presumes

The RM model's dimensions are not equally well-posed on every layer projection of Section 3.2. $FT(v)$ and $M(v)$ are continuous functionals of reachability and coupling and vary smoothly across projections of any size. $A(v)$ is different: its dominant terms, $\text{AP}_{c,\text{directed}}$ (directed articulation-point membership) and BR (bridge ratio), are discrete structural predicates that fire only where the projected graph actually contains cut vertices or bridge edges. A projection that happens to be densely interconnected—many alternate paths between any two applications—can have *zero* such structures by construction, in which case $A(v)$ collapses to a near-constant value for every vertex in that projection, regardless of the underlying system's true availability risk.

This is a property of the projection, not a defect in the formula: $A(v)$ is answering "does removing v disconnect this graph," and on a well-connected projection the honest answer is "no, for almost every v ." Because $A(v)$ carries $(1 - \alpha) = 0.64$ of $R(v)$'s weight and $R(v)$ carries $w_R = 0.80$ of $Q(v)$'s weight, a near-constant $A(v)$ on a given layer flattens roughly half of the composite score on that layer specifically—not a general property of the RM model, but a property of scoring it on a projection where the availability substrate is degenerate. Section 9.3 reports this concretely for the application-layer projection used throughout Section 9.1's headline evaluation. The practical implication is that Availability-focused review (**SPOF** and **BROKER_OVERLOAD** detection) should be scored on a layer projection where cut structures are structurally possible—the middleware or system layer (Section 3.2), where brokers and physical nodes reintroduce genuine bottlenecks—rather than assumed to transfer unchanged from the application layer.

4 A Catalog of Architectural Anti-Patterns for Publish–Subscribe Systems

4.1 Anti-Patterns vs. Bad Smells in Automated Review

In automated code review, findings fall into two distinct confidence tiers:

- **Architectural Anti-Pattern:** A proven structural pathology known to cause systemic vulnerability or failure cascades, requiring structural intervention (e.g., Single Point of Failure, God Component, Broker Overload).
- **Architectural Bad Smell:** A localized structural heuristic signaling potential technical debt, warranting review and localized refactoring (e.g., Chatty Pair, QoS Mismatch, Orphaned Topic).

Publish–subscribe architectural anomalies leave distinct **topological signatures** in the dependency graph. Because these signatures are computable from static descriptors, automated review bots can flag them before code is deployed.

4.2 Detection Methodology

Detection operates over the same fourteen structural inputs that feed RM attribution (Section 3.4): Reverse PageRank (RPR), in-degree and out-degree (DG_{in} , DG_{out}), multi-path coupling index (MPCI), topic fan-out criticality (FOC), betweenness (BT), QoS-weighted in- and out-degree (w_{in} , Topic-only, w_{out}), clustering coefficient (CC), path complexity (PC), directed articulation score ($AP_{c,directed}$), bridge ratio (BR), connectivity degradation index (CDI), and component weight (w)—plus the synthesized Code Quality Penalty (CQP, Section 3.3), for fifteen scoring metrics in total, verified field-for-field against the **SCORING** role in the codebase’s metric registry. Forward centralities (PageRank, Closeness, Eigenvector) feed only the learned GNN companion pathway of the JSS submission [13] and play no role in this deterministic detector layer. Detectors employ **adaptive box-plot thresholds** ($Q_3 + k$ IQR, fixed at $k = 1.5$ in this work; Section 11.2 discusses sweeping k).

4.3 Catalog Overview

Table 3 summarizes the nineteen anti-patterns and smells spanning three severity tiers, four catalog categories, and three finding scopes—**Component** (attributed to one vertex), **Edge** (attributed to one relationship), and **System** (a corpus-wide statistic, not attributable to any single element). The scope column matters for interpreting Section 9.2: component-level agreement metrics (precision, recall, Cohen’s κ) are measured against component-scoped findings, while edge-scoped findings only implicate components indirectly, as endpoints.

4.4 Representative Pattern Walkthroughs

1. **SPOF (Single Point of Failure)**: A node whose removal disconnects the graph ($AP_{c,directed}(v) > 0$). Detection combines graph connectivity testing with QoS weighting ($QSPOF(v) = AP_c(v) \times w(v)$). Remediation generates container anti-affinity constraints.
2. **GOD_COMPONENT**: An entity exhibiting extreme betweenness and CRITICAL maintainability ($BT(v) > 0.30 \wedge M(v) \in \text{CRITICAL}$). Remediation decomposes pub-sub channels via logical topic splitting.
3. **BROKER_OVERLOAD (Cautionary Benchmark Case)**: A broker routing $\geq 2\times$ the median broker’s traffic. *Cautionary finding*: In scenario S05 (two equally overloaded brokers), the within-population relative median rule cannot fire because each broker *is* the median. This highlights the necessity of absolute capacity-aware thresholds (Section 11.2).
4. **CHATTY_PAIR**: Two components maintaining bidirectional high-frequency dependencies across separate topics ($(u \rightarrow v) \wedge (v \rightarrow u)$), representing logical coupling masquerading as pub-sub decoupling.
5. **QOS_MISMATCH**: A publisher offering weaker reliability than the subscriber requires ($w_{pub}(u) < w_{sub}(v) - \tau$). In DDS/ROS 2, this produces silent connection dropouts at runtime without compile-time errors.

4.5 Detection Validation Methodology

Detection accuracy is validated against an independent discrete-event cascade simulation oracle ($I_{comp}(v)$) that models cascading message dropouts and queue exhaustion under component removal.

Table 3: Publish-Subscribe Architectural Anti-Pattern and Bad Smell Catalog. **Trig.** marks patterns that fire at least once on the eight-scenario benchmark corpus (Section 4.6).

Pattern	Sev.	Category	Scope	Detection Signal	Autom. Refactor- ing	Trig.
SPOF	CRITICAL	Availability	Component	Articulation point $\wedge$ QoS SPOF score	Anti-affinity reallocation	✓
SYSTEMIC_RISK	CRITICAL	Reliability	System	Share of CRITICAL components $> 20\%$	Advisory	
CYCLE	HIGH	Architecture	System	Strongly connected component / loop	Advisory	✓
GOD_COMPONENT	CRITICAL	Maintainability	Component	Extreme betweenness $\wedge$ CRITICAL $M(v)$	Logical topic splitting	
BOTTLENECK_EDGE	HIGH	Availability	Edge	Edge betweenness centrality outlier	Logical topic splitting	✓
BROKER_OVERLOAD	HIGH	Availability	Component	Broker availability $\geq 2\times$ median load	Anti-affinity reallocation	
DEEP_PIPELINE	HIGH	Reliability	System	Path length $\geq \max(5, P_{75})$	Advisory	(excl.)
TOPIC_FANOUT	MEDIUM	Reliability	Component	Topic subscriber out-degree outlier	Logical topic splitting	
CHATTY_PAIR	MEDIUM	Maintainability	Pair	Bidirectional weight product $> \tau_{chatty}$	Advisory	
QOS_MISMATCH	MEDIUM	Reliability	Edge	Pub/Sub QoS-weight gap $> \tau_{qos}$	Advisory (Manual / Relay)	✓
ORPHANED_TOPIC	MEDIUM	Maintainability	Component	Zero in- or out-degree on graph	Advisory	
UNSTABLE_INTERFACE	MEDIUM	Maintainability	Component	High CouplingRisk_enh $\wedge$ high $M(v)$	Advisory	✓
BRIDGE_EDGE	HIGH	Availability	Edge	Graph-theoretic bridge cut-edge	Advisory	✓
FAILURE_HUB	CRITICAL	Reliability	Component	Reliability outlier $\wedge$ high out-degree	Logical topic splitting	
CONCENTRATION_RISK	MEDIUM	Reliability	System	Top-3 PageRank share > 0.50	Advisory	
HUB_AND_SPOKE	MEDIUM	Maintainability	Component	Low clustering $\wedge$ degree > 3	Logical topic splitting	✓
CHAIN	MEDIUM	Architecture	System	Degree-bounded linear subgraph	Advisory	
ISOLATED	MEDIUM	Architecture	Component	Zero total degree	Advisory	✓
COMPOUND_RISK	CRITICAL	Architecture	Component	Co-occurring SPOF + God/Hub finding	Anti-affinity reallocation	

4.6 Catalog Coverage on the Benchmark Corpus

A catalog contribution should disclose which of its patterns are actually exercised by its own evaluation corpus, not only which are specified. Across all eight benchmark scenarios (Section 8.2), DEEP_PIPELINE is excluded for the reason given in Section 9.5, and of the remaining eighteen active detectors, exactly **eight trigger at least one finding** (Table 3, **Trig.** column): Bottleneck Dependency (BOTTLENECK_EDGE, 4974 findings across the corpus), QoS Policy Mismatch (QOS_MISMATCH, 1243), Unstable Interface (64), SPOF (17), Hub-and-Spoke (14), Dependency Cycle (CYCLE, 9), Bridge Edge (5), and Isolated Component (3). The remaining ten active patterns—including GOD_COMPONENT, FAILURE_HUB, BROKER_OVERLOAD, TOPIC_FANOUT, CHATTY_PAIR, ORPHANED_TOPIC, CONCENTRATION_RISK, CHAIN, SYSTEMIC_RISK, and COMPOUND_RISK—are specified and unit-tested but never fire on this corpus. This is consistent with, and explains, Section 7.1’s automation-footprint disclosure being scoped to what the corpus can exercise rather than what the catalog specifies; it does not indicate the unexercised patterns are miscalibrated, since triggering depends on topological structure this synthetic corpus may simply not contain (e.g., SYSTEMIC_RISK requires more than 20% of a scenario’s components at CRITICAL, which none of the eight scenarios reach).

Of the 6329 total findings the triggered eight produce, only **98 (1.5%)** are attributable to a single component (SPOF 17, Unstable Interface 64, Isolated Component 3, Hub-and-Spoke 14); the remaining **6231 (98.4%)** are Edge- or System-scoped (Bottleneck Dependency 4974 and QoS Policy Mismatch 1243 alone account for 6217 of these). This scope imbalance is the mechanism behind Section 9.2’s implication-rate and agreement findings: a catalog whose active findings are overwhelmingly non-component-scoped will still implicate a large fraction of components, as edge endpoints, even when its direct component-level agreement with ground truth is weak.

5 Closed-Loop Optimization Objective

The prescriptive engine consumes the Diagnostic Pathway’s output directly—severity tiers from Section 3.4 and pattern findings from Section 4—rather than re-deriving its own criticality signal, so diagnosis and prescription cannot silently drift apart. The prescriptive refactoring objective computes a graph transformation $\Delta(G)$ producing mutated topology $G' = \Delta(G)$ that minimizes global failure impact subject to a budget constraint:

$$\min_{\Delta} \sum_{v \in V} I_{\Delta(G)}^*(v) \quad \text{subject to} \quad \text{Cost}(\Delta) \leq \mathcal{B} \quad (11)$$

Candidate refactorings are compiled from entities flagged **CRITICAL** or **HIGH** by the Diagnostic Pathway. Rather than relying on an unconstrained heuristic search, SaG-Prescribe enforces a **two-level closed-loop acceptance filter**:

1. **Per-Edit Counterfactual Acceptance Filter:** Each candidate edit δ is applied in isolation to a sandboxed copy of G and simulated across propagation thresholds $\Theta = \{0.1, 0.2, 0.5\}$ and seed set $S = \{42, 123, 456\}$. It is retained if and only if its mean impact reduction exceeds the simulator’s seed noise at every threshold:

$$\overline{\Delta I}_{\theta}(\delta) > \kappa \cdot \sigma_{\text{seed}, \theta}(\delta) \quad \forall \theta \in \Theta \quad (12)$$

where $\kappa = 1.0$ is the noise-rejection margin.

2. **Whole-Policy Acceptance Gate:** The surviving subset of edits Δ_{admitted} is applied jointly to G , and the resulting system is re-evaluated. The whole policy is accepted if and only if the net System Risk Index improves:

$$\Delta \text{SRI} = \text{SRI}_{\text{baseline}} - \text{SRI}_{\text{mutated}} > 0 \quad (13)$$

6 The SaG-Prescribe Prescriptive Pipeline

6.1 Hexagonal Core Architecture

SaG-Prescribe implements a hexagonal ports-and-adapters architecture. The domain service depends on the `IGraphRepository` port:

- **Neo4jRepository:** Bolt-driven graph store for persistent, production deployments.
- **MemoryRepository:** High-speed, in-memory graph repository ensuring thread-safe, isolated sandboxing for counterfactual verification in CI/CD pipelines.

6.2 The Seven Pipeline Stages

1. **Stage 1 (Model Ingestion):** Ingest JSON/YAML Architecture-as-Code descriptors and file-level SCA metrics.
2. **Stage 2 (Topological Projection):** Construct multi-layer graph G_{analysis} and derived `DEPENDS_ON` edges.
3. **Stage 3 (Diagnostic Quality Attribution):** Compute CQP, RM dimensions (FT, A, R, M), and ISO 25019 harm mapping.
4. **Stage 4 (Anti-Pattern Detection):** Run the 19-pattern detection engine with adaptive box-plot thresholds.
5. **Stage 5 (Candidate Compilation):** Compile candidate refactoring mutations from `CRITICAL/HIGH` findings.
6. **Stage 6 (Counterfactual Verification):** Execute two-level simulation verification in `MemoryRepository`.
7. **Stage 7 (Review Interface):** Emit verified refactoring blueprint and CI/CD quality gate verdicts.

6.3 Refactoring Mutation Operators

1. **Logical Topic Splitting (Operator 1):** Targets `TOPIC_FANOUT`, `GOD_COMPONENT`, and `FAILURE_HUB`. Decomposes a multi-publisher topic T into dedicated sub-topics $\{T_a : a \in P(T)\}$, isolating data flows and bounding blast radius.
2. **Physical Anti-Affinity Reallocation (Operator 2):** Targets `SPOF`, `BROKER_OVERLOAD`, and `COMPOUND_RISK`. Generates orchestration anti-affinity constraints to migrate co-located critical components from shared compute host N_{from} to isolated host N_{to} , updating `RUNS_ON` and `CONNECTS_TO` edges.
3. **Transport QoS Contract Hardening (Operator 3):** Fires on `CRITICAL/HIGH` channels operating under volatile contracts (`BEST_EFFORT / VOLATILE`), upgrading them to `RELIABLE` and `TRANSIENT_LOCAL`.

Automation Footprint Disclosure. Exactly five of the nineteen patterns directly trigger an automated mutation operator (`SPOF` $\rightarrow$ Operator 2; `GOD_COMPONENT`, `BOTTLENECK_EDGE`, `FAILURE_HUB`, `HUB_AND_SPOKE` $\rightarrow$ Operator 1). Operator 3 triggers from generic criticality tiers. The remaining fourteen patterns provide advisory recommendations for human review. Of these five, two—`SPOF` and `BOTTLENECK_EDGE`—are among the eight patterns Section 4.6 finds actually trigger on the benchmark corpus; the other three (`GOD_COMPONENT`, `FAILURE_HUB`,

HUB_AND_SPOKE) are specified operator triggers this corpus does not exercise. In practice, Rule 2 (anti-affinity reallocation) fires from RM criticality tier membership and SPOF findings rather than the catalog name directly, while Rule 1 (topic splitting) fires on multi-publisher/multi-subscriber congestion, RM-critical topics, or a topic feeding a Bottleneck/Hub-matched finding—so the corpus’s candidate mix (Table 8) is shaped as much by criticality-tier membership as by which of the five triggering patterns fires.

6.4 Closed-Loop Counterfactual Verification Procedure

Candidate generation and verification are strictly decoupled: candidate refactorings are generated exclusively from G_{analysis} and diagnostic RM attribution, and edits are simulated in isolation on $G_{\text{structural}}$ within `MemoryRepository`, enforcing complete view independence.

7 DevOps Integration and CI/CD Quality Gating

7.1 Automated Code Review Bot Architecture

To govern architectural quality during continuous evolution, the Diagnostic Pathway is packaged as a lightweight pre-merge review bot. When a pull request modifies system topology or QoS configurations, the bot analyzes the change, executes the 19 anti-pattern detectors, attributes RM debt, and posts structured review comments.

7.2 Regression Semantics: Absolute vs. Delta-Aware Gating

- **Absolute Gating:** Evaluates the entire topology against global thresholds. However, real-world industrial systems carry intentional, risk-accepted legacy debt; blocking builds on pre-existing debt paralyzes development.
- **Delta-Aware Gating (Design):** Compares the PR candidate topology against the target branch merge-base, blocking only *newly introduced* anti-patterns or worsening severities, while honoring a signed **waiver register**. Absolute mode is implemented today; delta-aware mode is specified here for production integration but not yet implemented.

Absolute Gating Outcome on the Benchmark Corpus. Every one of the eight benchmark scenarios contains at least one CRITICAL- or HIGH-severity finding (Section 9.2), so under absolute gating the review bot returns **Exit Code 2 (Block)** on all eight scenarios. Given the 94.8% mean component-implication rate this catalog configuration produces (Section 9.1), this is the expected, mechanical consequence of absolute thresholds applied globally, not a scenario-specific anomaly—and it is precisely why delta-aware gating is a necessity for adoption rather than a convenience: an absolute gate that blocks every commit on legacy debt trains developers to ignore or bypass it.

7.3 Three-Tier Exit-Code Protocol

- **Exit Code 0 (Pass):** No findings above threshold; deployment permitted.
- **Exit Code 1 (Warning):** MEDIUM-severity smells detected; build passes with advisory comments.
- **Exit Code 2 (Block):** CRITICAL or HIGH anti-patterns detected; build fails, blocking merge.

8 Experimental Design

8.1 Research Questions

- **RQ1 (Diagnostic Ranking Efficacy):** How does the RM composite’s rank correlation with cascade failure impact compare to single-metric structural baselines (betweenness, degree)?
- **RQ2 (Catalog Screening and Calibration):** How well does the anti-pattern catalog’s component-level agreement match its precision/recall profile, and what accounts for its implication rate?
- **RQ3 (Attribution Validity and Substrate Adequacy):** Does the Availability dimension carry genuine signal on the layer this evaluation measures, how does stratification by node type affect the pooled ranking result, and how does this re-measurement compare to our prior published claim?
- **RQ4 (Prescriptive Refactoring Efficacy & Operator Survival):** Does the closed-loop prescriptive engine achieve statistically significant reductions in System Risk Index across heterogeneous scenarios, and how do individual refactoring operators survive counterfactual verification?
- **RQ5 (Verification Yield, Compositionality & CI/CD Feasibility):** What proportion of generated candidates survive verification, do individually admitted edits compose beneficially, and what is the execution latency of diagnostic gating and prescriptive verification across system scales?

8.2 Benchmark Scenarios

We evaluate eight benchmark scenarios spanning six architectural domains—autonomous vehicles, IoT smart cities, financial trading, healthcare, hub-and-spoke microservices, and hyper-scale enterprise systems—plus a minimal regression fixture (Table 4). Scenarios S01–S08 evaluate detection; S01–S07 evaluate prescription (S08 excluded as a minimal regression smoke test, not as a scale-driven exclusion).

Table 4: Benchmark Scenario Scales and Topological Dimensions.

Scenario	Applications	Libraries	Topics	Brokers	Nodes	Total Edges ($ E $)
S01 Autonomous Vehicle	80	20	40	4	8	797
S02 IoT Smart City	200	10	80	6	30	1322
S03 Financial Trading	60	18	35	5	6	580
S04 Healthcare	50	12	25	3	8	400
S05 Hub-and-Spoke	70	25	30	2	12	797
S06 Microservices Mesh	90	30	45	6	15	680
S07 Hyper-Scale Enterprise	300	50	120	10	40	3245
S08 Tiny Regression	12	4	8	2	3	101

8.3 Experimental Protocol and Metrics

- **Detection Metrics:** Spearman rank correlation $\rho(Q, I_{\text{comp}})$ (pooled and stratified), Cohen’s κ , NDCG@10, Top- k overlap, precision, recall, and F_1 against simulation ground truth I_{comp} (Section 4.5). Baselines: betweenness and degree centrality. For $Q(v)$ /betweenness/degree, the predicted and true critical sets are both sized by the same box-plot-or-top-20% rule (Section 9.1), which pins precision, recall, and F_1 to near-equal values by construction;

the catalog’s flagged set has no such constraint, so its precision and recall are measured independently.

- **Prescriptive Metrics:** System Risk Index ($SRI = 0.5(1 - H_R) + 0.5(1 - H_M)$), $\Delta SRI = SRI_{\text{baseline}} - SRI_{\text{mutated}}$, per-operator candidate generation counts, admitted edit counts, and per-edit survival rates.
- **Verification Parameters:** $\kappa = 1.0$, $\Theta = \{0.1, 0.2, 0.5\}$, $S = \{42, 123, 456\}$.
- **Two Oracles, Not One:** This paper’s ground truth, $I_{\text{comp}}(v)$, is an independent discrete-event cascade simulator modeling message dropouts and queue exhaustion under component removal on the directed **DEPENDS_ON** projection. Our prior work [12] validated against a different oracle, Reachability Loss, on the undirected pub-sub graph—a distinction Section 9.3 returns to when reconciling the two papers’ correlation figures.

9 Empirical Results

9.1 Diagnostic Ranking Efficacy (RQ1)

Table 5 presents detection validation at the application layer across all eight scenarios; Table 6 compares the three ranking predictors across four independent quality measures.

Table 5: Diagnostic Detection Validation (App Layer) against Simulation Ground Truth $I_{\text{comp}}(v)$.

Scenario	n	$\rho(Q, I)$	F_1	Top-5	Catalog P	Catalog R	Implicated %	Gate (s)
S01 Autonomous Vehicle	80	0.457	0.600	0.40	0.244	0.950	97.5%	0.59
S02 IoT Smart City	200	0.718	0.780	0.40	0.310	0.880	71.0%	0.56
S03 Financial Trading	60	0.365	0.667	1.00	0.237	0.933	98.3%	0.40
S04 Healthcare	50	0.451	0.538	0.60	0.213	0.769	94.0%	0.16
S05 Hub-and-Spoke	70	0.466	0.611	0.80	0.257	1.000	100.0%	0.92
S06 Microservices Mesh	90	0.405	0.565	0.40	0.261	1.000	97.8%	0.35
S07 Hyper-Scale Enterprise	300	0.480	0.627	0.20	0.251	1.000	99.7%	20.98
S08 Tiny Regression	12	0.538	0.333	0.60	0.250	1.000	100.0%	0.01
Mean	—	0.485	0.590	0.550	0.253	0.942	94.8%	3.00

Table 6: Predictor Comparison, App Layer, Mean over 8 Scenarios. No predictor dominates on every measure.

Predictor	ρ	Cohen’s κ	NDCG@10	Top-10
$Q(v)$ composite (RM)	0.4850	0.4514	0.8295	0.5875
Betweenness	0.4346	0.4139	0.8592	0.6250
Degree	0.5187	0.4792	0.8214	0.5625

Key Diagnostic Findings.

1. **Ranking Saturation, Not a Clear Winner.** Table 6 shows all three predictors cluster tightly: degree centrality leads on ρ (0.519) and κ (0.479), betweenness leads on NDCG@10 (0.859) and Top-10 overlap (0.625), and the RM composite is never last by more than 0.03 on any measure. No predictor is uniformly better, which means the composite’s contribution cannot rest on out-ranking a scalar centrality score—it does not. What the RM composite adds beyond ranking is the decomposition Section 3.4 defines: a scalar centrality count has no Fault Tolerance/Availability/Maintainability breakdown to route to a Reliability Engineer, an SRE, or an Architect.

2. **Ranking Improves at Scale.** Stratifying by scenario size, mean ρ is 0.599 for the two large scenarios (≥ 150 components: S02, S07) versus 0.447 for the six smaller ones—the composite’s ranking quality is not degrading as systems grow, which is the direction that matters for the CI/CD use case this paper targets.

3. **Catalog Screening Behavior.** The anti-pattern catalog functions as a sensitive screen: mean recall is 0.942, precision is 0.253, implicating a mean of 94.8% of components at CRITICAL/HIGH severity. Section 9.2 explains the mechanism. Note that for the $Q(v)$ /betweenness/degree predictors, precision, recall, and F_1 are numerically identical (Section 8.3)—an artifact of the equal-sized-set validation rule, not evidence that these predictors have no false positives or false negatives individually.

9.2 Catalog Screening and Calibration (RQ2)

Precision and recall alone understate how the catalog’s findings relate to ground-truth criticality. Cohen’s κ measures chance-corrected agreement between the catalog’s flagged/unflagged partition and the true critical/non-critical partition; it is $\kappa = -0.0022$ at the application layer (mean over 8 scenarios, range -0.125 to $+0.141$) and $\kappa = -0.0300$ at the system layer—both indistinguishable from chance agreement, despite recall of 0.942.

The mechanism is scope, not miscalibration of any single threshold: Section 4.6 shows that of the 6329 findings the eight triggered patterns produce across all scenarios, only 98 (1.5%) are attributable to a single component, while 6231 (98.4%) are Edge- or System-scoped. `n_flagged_directly`—findings whose entity is a component—is zero in 4 of 8 scenarios. A component is “implicated” almost entirely by being an endpoint of a flagged edge (chiefly BOTTLENECK_EDGE and QOS_MISMATCH), not by a component-level detector firing on it. High recall together with chance-level κ is therefore consistent, not contradictory: the catalog reliably flags *some* edge touching almost every genuinely critical component (hence high recall), while its component-level partition of flagged vs. unflagged carries almost no discriminative information beyond what a random partition of the same size would (hence $\kappa \approx 0$). We report this as a calibration finding: the catalog, as configured with a fixed $k = 1.5$ IQR fence (Section 4.2), is a highly sensitive but poorly discriminating component-level screen, and its practical CI/CD value depends on the delta-aware gating design of Section 7.2, not on tightening k alone—since tightening k would improve κ at recall’s expense on the same edge-dominated finding mix.

9.3 Attribution Validity and Substrate Adequacy (RQ3)

The Availability Dimension Is Degenerate at This Layer. Section 3.5 anticipates that $A(v)$ requires a projection where cut vertices and bridge edges actually occur. At the application layer, they largely do not: the SPOF detector scores $F_1 = 0.0$ in **all 8 scenarios**, predicting zero articulation points in 6 of 8 (the two exceptions, S02 and S04, predict 1–4 apiece against ground truth that is itself near-zero). Since $A(v)$ carries 0.64 of $R(v)$ ’s weight and $R(v)$ carries 0.80 of $Q(v)$ ’s weight, this means roughly half of the composite score is measured on a substrate with almost no structural variance at this layer—a concrete instance of the mechanism Section 3.5 states as a design-time caveat, not a failure discovered post hoc. This is also a plausible contributor to Table 6’s saturation result: a composite that devotes half its weight to a near-constant term should be expected to under-perform a predictor (degree) that has no such dead weight.

Reconciling with Our Prior Published Result. Section 1.5 previewed the delta with [12]’s $\rho = 0.94$; the mechanism above completes the explanation. That prior result used (a) Reachability Loss as ground truth, a connectivity statistic computed on the same undirected application graph its $CS(v)$ predictor is computed on—predictor and oracle share a substrate, which

structurally favors high correlation; (b) a single ten-application system, where the specific mix of articulation points present is a property of that one topology, not a population statistic; and (c) $AP(v)$ contributing 30% of the composite by design, which is informative precisely because that topology had articulation points to find. This paper’s I_{comp} oracle is an independent discrete-event cascade simulator sharing no computational substrate with the RM score, is measured across eight topologies up to 300 applications, and is measured on a directed `DEPENDS_ON` projection where—as shown above—articulation structure is largely absent. Both results are correct measurements of what they measured; they are not measurements of the same underlying quantity, and the honest reading is that ranking correlation against a connectivity-derived oracle is not a portable measure of a criticality score’s real-world diagnostic value.

Simpson’s Paradox in Node Pooling. At the system layer (pooling all five node types), the pooled correlation is $\rho = 0.028$, while per-type strata show robust positive correlations (Application 0.503, Broker 0.395, Node 0.142)—the pooled figure falls *outside* the per-type range entirely (`pooled_inside_per_type_range: false`). This is a severe Simpson’s paradox from between-type score offsets: node types with systematically different score levels but internally consistent rankings can pool into a near-zero aggregate correlation. Stratified evaluation is therefore not a presentational choice but a methodological requirement for heterogeneous software graphs; every correlation this paper reports outside this paragraph is single-type (Application) or explicitly stratified for this reason.

9.4 Prescriptive Refactoring Efficacy and Operator Survival (RQ4)

Table 7 presents prescriptive refactoring results under per-edit counterfactual verification across all seven benchmark scenarios; Table 8 breaks this down by refactoring operator.

Table 7: Prescriptive Refactoring Performance under Per-Edit Counterfactual Verification.

Scenario	Base SRI	Mut SRI	Δ SRI	Candidates	Admitted	Rejected	Admit %
S01 Autonomous Vehicle	0.3246	0.3160	+0.0086	156	45	111	28.8%
S02 IoT Smart City	0.3749	0.3288	+0.0461	391	297	94	76.0%
S03 Financial Trading	0.3409	0.3301	+0.0108	125	22	103	17.6%
S04 Healthcare	0.3394	0.3285	+0.0109	99	36	63	36.4%
S05 Hub-and-Spoke	0.3223	0.3108	+0.0115	127	107	20	84.3%
S06 Microservices Mesh	0.3293	0.3196	+0.0097	163	142	21	87.1%
S07 Hyper-Scale Enterprise	0.3283	0.3112	+0.0171	528	479	49	90.7%
Total / Overall	—	—	—	1589	1128	461	71.0%

Table 8: Candidate Generation vs. Verified Admission by Refactoring Operator.

Refactoring Operator	Candidates	Admitted	Survival Rate	Primary Structural Target
Physical Anti-Affinity Reallocation	1188	923	77.7%	Single points of failure, co-located components
Transport QoS Contract Hardening	69	40	58.0%	Volatile transport contracts over unreliable links
Logical Topic Splitting	332	165	49.7%	High-fan-out message channels
Total	1589	1128	71.0%	—

Key Prescriptive Findings.

- 1. Statistically Significant Improvement:** Every scenario achieves a net positive Δ SRI. A Wilcoxon signed-rank test against zero yields $W = 0, p = 0.0156 (n = 7)$, confirming statistically significant risk reduction across all evaluated domains.

2. **Substantial Yield:** Across 1589 generated candidates, **1128 edits** (71.0%) survive counterfactual verification.
3. **Anti-Affinity Reallocation Demonstrates the Highest Survival Rate (77.7%),** achieving near-100% survival in dense topologies (408/409 in Enterprise, 123/123 in Microservices)—mechanistically plausible, since Rule 2 only fires on nodes already flagged SPOF or RM-critical (Section 6.3), pre-filtering its candidate pool to genuine structural risk. Transport QoS hardening achieves 58.0% survival where triggered, while topic splitting achieves 49.7%.

9.5 Verification Yield, Compositionality & CI/CD Feasibility (RQ5)

1. **Informative Rejection:** All 461 rejected edits record the binding propagation threshold θ where impact reduction failed to clear $\kappa\sigma_{\text{seed}}$, providing transparent review feedback.
2. **Compositional Dynamics:** While every scenario achieves net positive system-level ΔSRI , two scenarios (S03 Financial Trading and S06 Microservices Mesh) exhibit slight negative mean per-component changes among accepted edits. This demonstrates that individually verified edits can interact sub-additively, validating the necessity of the second-level whole-policy gate.
3. **Diagnostic Review Gate Latency:** The complete diagnostic pipeline (graph construction, CQP computation, RM attribution, and 18 active anti-pattern detectors) executes in **0.01 s to 20.98 s** (Table 5). The eighteen detectors run in under **0.2 s total**; computational overhead is dominated by base metric computation.
4. **DEEP_PIPELINE Bounding:** The unconstrained DEEP_PIPELINE detector attempts exhaustive path enumeration, generating 247,761 findings on tiny fixtures and hanging on larger graphs. Excluding this detector (Section 4.6) restores sub-second gate execution.
5. **Prescriptive Verification Latency:** The full 7-scenario prescriptive sweep (14,301 simulation sweeps) completed in **2 h 47 min** using multi-core process parallelism (20 workers), establishing its viability for nightly or on-demand CI/CD refactoring runs.

10 Discussion and Threats to Validity

10.1 Diagnostic Explainability vs. Threshold Calibration

The Diagnostic Pathway successfully provides deterministic, role-specific quality attribution (Reliability Engineer vs. SRE vs. Architect), translating abstract topological risk into actionable review decisions—and Section 9.1 shows this attribution, not superior ranking, is where its value over a scalar centrality score lies. That value is not free, however: Section 9.2 shows the catalog’s fixed $k = 1.5$ IQR fence produces near-chance component-level agreement ($\kappa \approx 0$) despite high recall, because 98.4% of active findings are edge-scoped and implicate components only as endpoints. Tightening k would trade recall for precision on the same finding mix, but would not by itself change the underlying scope imbalance—a catalog dominated by two edge-level detectors (BOTTLENECK_EDGE, QOS_MISMATCH) will always implicate components indirectly at scale. The mitigation this paper actually validates is architectural rather than purely statistical: absolute gating blocks on every one of the eight benchmark scenarios (Section 7.2), which is unusable in practice, while delta-aware gating—blocking only newly introduced findings relative to a merge-base—bounds the review surface to what a given change actually introduces, regardless of the base rate. Threshold recalibration (Section 11.2) and delta-aware gating are therefore complementary, not substitutes: one narrows what fires, the other narrows what is enforced.

10.2 The Verify-Before-Recommend Discipline

Counterfactual verification proves essential: an unverified engine would have applied 461 harmful or ineffective mutations (29.0%). By filtering mutations against cascade noise, SaG-Prescribe ensures that recommended refactorings reliably harden system architecture. Notably, this discipline is what makes Section 9.4’s reallocation result trustworthy rather than merely plausible: the mechanistic argument for why anti-affinity reallocation should work (removing SPOF co-location) is exactly the kind of confident-sounding reasoning an open-loop recommender would have shipped regardless of whether it was true, and it is only because every candidate is independently simulated that the 77.7% survival rate is a measurement rather than an assumption.

10.3 Construct Validity: Substrate Adequacy

Beyond the general dependence on a discrete-event cascade simulator, this paper surfaces a more specific construct-validity threat, promoted here to first-class status rather than left implicit: a composite score’s sub-dimensions can be individually well- or ill-posed on a given graph projection, and scoring on an ill-posed projection silently degrades the composite without changing its formula. Section 3.5 states this as a design-time property; Section 9.3 measures its concrete instance—the Availability dimension carrying almost no variance on the application-layer projection used throughout this paper’s headline detection evaluation, because that projection has essentially no articulation points to find. The mitigation is not a different formula for $A(v)$, but scoring Availability-sensitive review on a projection where cut structures can occur (the middleware or system layer), and reporting per-layer results rather than treating the application layer as universally representative. We flag this as a threat other RM-style composite scores on typed dependency graphs should check for explicitly, not only fix locally: any weighted sum over structurally heterogeneous sub-scores inherits this risk whenever one term is a discrete graph predicate.

10.4 Comparison with Prior Self-Reported Results

Section 9.3 reconciles this paper’s $\rho = 0.485$ against our own prior published $\rho = 0.94$ [12]. In discussion terms, the general lesson is broader than this one paper pair: a criticality score whose ground-truth oracle is itself a connectivity statistic on the same graph the score is computed from will tend to correlate well with that oracle almost by construction, and that correlation does not necessarily transfer to an oracle with independent causal mechanics (here, a discrete-event message-dropout and queue-exhaustion simulator). We consider the re-measurement a strength of this submission’s methodology rather than a weakness to minimize: reporting a negative result against one’s own prior work, with the mechanism identified rather than merely disclosed, is what makes the finding actionable for other researchers evaluating structural criticality scores against connectivity-derived oracles.

10.5 Further Threats to Validity

- **Construct Validity:** Both detection and verification utilize a discrete-event cascade simulator. We mitigate simulator dependency by grounding operators in established distributed systems dependability patterns (anti-affinity scheduling, topic isolation, QoS hardening), and by validating the composite against SPOF ground truth directly (Section 9.3) rather than relying on ranking correlation alone.
- **Internal Validity:** Decoupled in-memory repositories ensure no circular data leakage between diagnostic candidate generation and counterfactual simulation.

- **External Validity:** Scenarios are synthetic domain models; validation on harvested industrial cyber-physical codebases remains high-priority future work.
- **Conclusion Validity:** The Wilcoxon test underlying Section 9.4’s significance claim has $n = 7$, the minimum for which a two-sided signed-rank test can reach $p < 0.05$; the reported $p = 0.0156$ is the best attainable significance at this sample size, not evidence of an especially large effect.

11 Conclusion and Future Work

11.1 Conclusion

This paper presented **SaG-Prescribe**, bringing the **Diagnostic Pathway** to the forefront of automated code review and software quality evaluation for distributed publish–subscribe systems. By integrating code-level SCA metrics (CQP), multi-layer heterogeneous dependency modeling, hierarchical ISO 25010/25019 RM quality attribution, and formal anti-pattern detection with counterfactually verified refactoring, SaG-Prescribe closes the Architecture–Code Gap. Rather than claiming diagnostic superiority in ranking accuracy, the paper establishes the diagnostic contribution empirically as *attribution and calibration evidence*: three structural predictors saturate around comparable ranking quality (Section 9.1), the anti-pattern catalog is a sensitive but chance-level-agreeing component-level screen whose mechanism we identify (Section 9.2), and the Availability dimension’s substrate adequacy is stated as a modeling property and measured directly rather than assumed (Section 9.3)—including an explicit, mechanistically explained re-measurement of our own prior published criticality-ranking claim. Empirical results further confirm that diagnostic review gating executes in sub-second to 20s latency, while counterfactual verification reliably admits 71.0% of refactoring candidates, delivering statistically significant resilience improvements across all evaluated domains.

11.2 Future Work

1. **Detection Threshold Recalibration:** Sweep the box-plot fence multiplier k in $Q_3 + k \text{ IQR}$ —already a parameter of `BoxPlotClassifier`, but hardcoded at $k = 1.5$ in the anti-pattern detectors (Section 4.2)—against the precision/ κ /recall trade-off Section 9.2 measures, rather than assuming the classical outlier convention is the right operating point for a CI/CD screen.
2. **Layer-Aware Availability Scoring:** Score $A(v)$ on a projection where cut structures are structurally possible (Section 3.5), and report Reliability/Availability results per-layer by default rather than defaulting to the application layer alone.
3. **Path-Bounded DEEP_PIPELINE:** Restrict path enumeration to bound execution time.
4. **Delta-Aware CI/CD Gating:** Implement merge-base diffing and waiver registers.
5. **Combinatorial Subset Verification:** Explore greedy forward-selection to optimize multi-edit composition.
6. **Dynamic κ Estimation:** Adapt noise margins based on empirical simulator variance.
7. **Per-Candidate Latency Profiling:** Instrument fine-grained verification timing.
8. **Operator Attribution Ablation:** Isolate per-operator ΔSRI contributions.
9. **Catalog Extension for Hybrid Architectures:** Incorporate REST/gRPC hybrid microservice patterns.

10. **Industrial Telemetry Replication:** Validate against live ROS 2 and DDS production traces, including a direct replication of [12]’s reachability-loss validation on the larger corpus used here.
11. **LLM-Assisted Pull-Request Synthesis:** Integrate generative code models to automatically draft pull requests implementing verified refactoring blueprints.

Declarations

Funding: This work was supported in part by the Scientific and Technological Research Council of Turkey.

Competing Interests: The authors declare that they have no competing financial or non-financial interests that are directly or indirectly related to the work submitted for publication.

Data Availability: Replication code, scenario topologies, seeds, and execution logs are reproducible via the repository harness:

```
PYTHONPATH=. python reproduce/detection_validation.py --layer app \
--output results/detection_validation_app.json
PYTHONPATH=. python reproduce/run_prescribe_all.py --kappa 1.0
```

Ethics Approval: Not applicable.

References

- [1] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, “Towards automating code review activities,” *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 3156–3173, 2022.
- [2] S. Lee, H.-S. Park, J. Chae, and K.-J. Park, “Probabilistic latency analysis of the data distribution service in ros 2,” *IEEE Transactions on Industrial Informatics*, vol. 21, no. 3, pp. 2415–2426, 2025.
- [3] S. Lee, J. Kang, and K.-J. Park, “Dependency chain analysis of ros 2 dds qos policies: From lifecycle tutorial to static verification,” *IEEE Internet of Things Journal*, vol. 12, no. 4, pp. 3890–3901, 2025.
- [4] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [5] W. H. Brown, R. C. Malveau, H. W. McCormick, and T. J. Mowbray, *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. John Wiley & Sons, 1998.
- [6] G. Suryanarayana, G. Samarthiyam, and T. Sharma, *Refactoring for Software Design Smells: Managing Technical Debt*. Morgan Kaufmann, 2014.
- [7] D. Taibi, V. Lenarduzzi, and C. Pahl, “Microservices anti-patterns: A taxonomy,” in *Microservices: Science and Engineering*. Springer, 2020, pp. 211–228.
- [8] M. Harman, S. A. Mansouri, and Y. Zhang, “Search-based software engineering: Trends, techniques and applications,” *ACM Computing Surveys*, vol. 45, no. 1, pp. 1–61, 2012.
- [9] A. Aleti, B. Buhnova, L. Grunske, A. Koziolk, and I. Meedeniya, “Software architecture optimization methods: A systematic literature review,” *IEEE Transactions on Software Engineering*, vol. 39, no. 5, pp. 658–683, 2013.

- [10] ISO/IEC, “Iso/iec 25010:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model,” International Organization for Standardization, Tech. Rep., 2023.
- [11] —, “Iso/iec 25019:2023 systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model,” International Organization for Standardization, Tech. Rep., 2023.
- [12] I. O. Yigit and F. Buzluca, “A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems,” in *Proc. IEEE International Conference on Recent Advances in Systems Science and Engineering (RASSE)*, 2025.
- [13] Authors, “Software-as-a-graph: A static system analysis framework for pre-deployment quality gating and failure simulation of publish-subscribe middleware,” *Journal of Systems and Software*, 2026, under review.
- [14] S. Pallickara, H. Bulut, and G. Fox, “Fault-tolerant reliable delivery of messages in distributed publish/subscribe systems,” in *Proc. 4th IEEE International Conference on Automatic Computing (ICAC)*, 2007, pp. 12–21.
- [15] T. Chang, S. Duan, H. Meling, S. Peisert, and H. Zhang, “P2s: A fault-tolerant publish/subscribe infrastructure,” in *Proc. 8th ACM International Conference on Distributed Event-Based Systems (DEBS)*, 2014, pp. 198–207.
- [16] G. Wang, J. Koshy, S. Subramanian, K. Paramasivam, M. Zadeh, N. Narkhede, J. Rao, J. Kreps, and J. Stein, “Building a replicated logging system with apache kafka,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1654–1655, 2015.
- [17] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, A. Blanco, C. Clement, D. Drain, D. Jiang, D. Tang, G. Li, L. Zhou, D. Jiang, M. Zhou, and N. Duan, “Codexglue: A machine learning benchmark dataset for code understanding and generation,” in *Proc. NeurIPS Datasets and Benchmarks*, 2021.
- [18] T. Menzies and F. Rahman, “Bad smells in software analytics papers,” *IEEE Software*, vol. 35, no. 4, pp. 100–103, 2018.
- [19] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [20] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [21] E. AlOmar, M. W. Mkaouer, and A. Ouni, “Can refactoring be self-affirmed? an exploratory study on how developers document their refactorings,” *Empirical Software Engineering*, vol. 26, no. 6, p. 121, 2021.
- [22] G. Bavota, B. De Carluccio, R. Oliveto, M. Di Penta, and A. De Lucia, “When does a refactoring induce bugs? an empirical study,” in *Proc. IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2012, pp. 104–113.
- [23] A. Ouni, M. Kessentini, H. Sahraoui, and M. S. Hamdi, “Search-based web service refactoring using quality of service and code quality metrics,” *IEEE Transactions on Services Computing*, vol. 10, no. 4, pp. 636–649, 2017.
- [24] C. White, S. Agarwal, and H. Zhang, “Toward an understanding of large language models for code refactoring,” *IEEE Software*, vol. 41, no. 2, pp. 48–56, 2024.

- 838 [25] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang,
839 “Large language models for software engineering: A systematic literature review,” *ACM*
840 *Transactions on Software Engineering and Methodology*, vol. 33, no. 5, pp. 1–68, 2024.
- 841 [26] A. M. Al-Kaswan, T. Ahmed, M. Izadi, A. A. Sawant, P. Devanbu, and A. van Deursen,
842 “Automatic refactoring using large language models: A study on extract method and rename,”
843 *Empirical Software Engineering*, vol. 29, no. 4, p. 98, 2024.
- 844 [27] L. C. Freeman, “A set of measures of centrality based on betweenness,” *Sociometry*, vol. 40,
845 no. 1, pp. 35–41, 1977.
- 846 [28] A. Bakhtin, M. Esposito, V. Lenarduzzi, and D. Taibi, “Network centrality as a new
847 perspective on microservice architecture,” in *Proc. IEEE International Conference on*
848 *Software Architecture (ICSA)*, 2025, pp. 72–83.
- 849 [29] D. H. M. Falci, O. A. Gomes, and F. S. Parreiras, “Complex networks analysis for software
850 architecture: An hibernate call graph study,” *IEEE Access*, vol. 6, pp. 62 145–62 155, 2018.
- 851 [30] C. Y. Baldwin and K. B. Clark, *Design Rules, Volume 1: The Power of Modularity*. MIT
852 Press, 2000.
- 853 [31] C. J. Colbourn, *The Combinatorics of Network Reliability*. Oxford University Press, 1987.
- 854 [32] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*.
855 McGraw-Hill, 1980.